UI polish and testing

Theming, layout, forms, motion, accessibility, and tests

Dinu-Ștefan Rusu
FILS, Universitatea Politehnica București · Autumn 2026

TODAY

What you should leave with

One theme, two modes

Build the app's colors and type once with ThemeData, and get dark mode for free.

Forms that behave

Validators, focus order, keyboard types, and error messages a user can act on.

Reachable by everyone

Semantics, contrast, tap targets, text scaling, and localized strings.

Tests you trust

Unit, widget, bloc, golden and integration tests, all running on every pull request.

PART 1 · THEMING

Making it look like one app

ThemeData, Material 3, dark mode, and responsive layout

Colors do not belong inside widgets

```
// Repeated in forty widgets:
Container(color: const Color(0xFF3F51B5))
Text('Total', style: TextStyle(
  fontSize: 16, color: Colors.black87))

// Declared once, read everywhere:
final cs = Theme.of(context).colorScheme;
Container(color: cs.primaryContainer)
Text('Total', style: tt.titleMedium)
```

The first version has no dark mode.
Every hard-coded color is a value that cannot change with the platform, the user's settings, or next year's brand.
It is also unsearchable. Nobody can answer "which blue is the button" without reading forty files.

Read a role, never write a color. A widget that names `cs.primary` keeps working when the palette, the mode or the brand changes.

ThemeData and ColorScheme.fromSeed

```
ColorScheme _scheme(Brightness b) => ColorScheme.fromSeed(
      seedColor: const Color(0xFF3F51B5), brightness: b);

ThemeData appTheme(Brightness b) => ThemeData(colorScheme: _scheme(b));

// Read it in any widget, never re-declare it:
final cs = Theme.of(context).colorScheme;
final tt = Theme.of(context).textTheme;
```

One seed color produces the whole palette. Material 3 derives every role from it and keeps the contrast between a color and its `on` pair correct in both modes.

THEMING

The color roles you will actually use

ROLE	WHAT IT PAINTS	TEXT ON IT
primary	the main action: a filled button	onPrimary
primaryContainer	a tinted area for that action	onPrimaryContainer
surface	page background, cards, sheets	onSurface
secondary, tertiary	accents that are not the main action	onSecondary
error	validation and destructive actions	onError
outline	borders, dividers, disabled edges	

Every role has an `on` pair. Paint with one, write with the other, and contrast is already handled.

Dark mode is a second ThemeData

```
MaterialApp(
  theme: appTheme(Brightness.light),
  darkTheme: appTheme(Brightness.dark),
  themeMode: ThemeMode.system,   // or the user's saved choice
  home: const HomeScreen(),
);

// Rare, and usually a smell:
final isDark = Theme.of(context).brightness == Brightness.dark;
```

You do not design dark mode twice. If widgets read roles instead of colors, the same widget tree is already correct in both modes.

Typography and component themes

```
ThemeData(
  colorScheme: _scheme(b),
  fontFamily: 'Inter',
  appBarTheme: const AppBarTheme(centerTitle: true),
  inputDecorationTheme: const InputDecorationTheme(
    border: OutlineInputBorder(),
  ),
  filledButtonTheme: FilledButtonThemeData(
    style: FilledButton.styleFrom(minimumSize: const Size.fromHeight(48)),
  ),
);
```

The scale runs from `displayLarge` to `labelSmall`. Most screens need only three of its roles.

MediaQuery: what the window is doing

```
final size = MediaQuery.sizeOf(context);
final pad = MediaQuery.paddingOf(context);
final keyboard =
    MediaQuery.viewInsetsOf(context).bottom;
final scaler =
    MediaQuery.textScalerOf(context);
```

Ask for one metric, not all of them. `MediaQuery.of(context)` rebuilds your widget when any metric changes, including the keyboard opening. The `sizeOf` family rebuilds only on the value you read. `viewInsets` is the keyboard. `padding` is the notch and the home indicator.

MediaQuery describes the window, not your widget. It reports what the system is doing to the app: the size, the notch, the keyboard, and the user's text scale.

LayoutBuilder, breakpoints, and SafeArea

```
LayoutBuilder(builder: (context, constraints) {
  if (constraints.maxWidth < 600) return const TodoList();
  return const Row(children: [
    Expanded(flex: 2, child: TodoList()),
    Expanded(flex: 3, child: TodoDetail()),
  ]);
});

SafeArea(child: body);  // keeps content clear of notch and gestures
```

Break on the width you were given, not on the device. A phone in landscape, a tablet, and a split-screen window can all be the same width. Two breakpoints, near 600 and near 840, cover almost everything.

PART 2 · FORMS

Getting data out of a person

Form, TextFormField, validators, focus, and error display

The shape of a Flutter form

```dart
final _formKey = GlobalKey<FormState>();

Form(
  key: _formKey,
  autovalidateMode: AutovalidateMode.onUserInteraction,
  child: Column(children: [emailField, passwordField, submitButton]),
);

if (_formKey.currentState!.validate()) await _signIn();
```

The Form is a coordinator, not a container. It finds every descendant field, runs their validators, and reports one boolean. The fields keep their own state.

Validators that say what to fix

```
String? validateEmail(String? v) {
  final s = v?.trim() ?? '';
  if (s.isEmpty) return 'Enter your email';
  if (!s.contains('@')) {
    return 'This address is missing an @';
  }
  return null;   // null means valid
}
```

Return null when the value is fine. Any other string becomes the red line under the field. One message, naming the one thing to change. ”Invalid input” tells the user nothing. Trim before you check, and never reject a password for containing a space.

A validator is a pure function. A string in, a string or null out. That makes it the easiest thing in the whole app to unit test, as Part 5 shows.

Keyboard, focus order, and submit

```
TextFormField(
  controller: _email,
  focusNode: _emailFocus,
  keyboardType: TextInputType.emailAddress,
  textInputAction: TextInputAction.next,
  onFieldSubmitted: (_) => _passwordFocus.requestFocus(),
  decoration: const InputDecoration(labelText: 'Email'),
);

// In dispose(): _email.dispose(); _emailFocus.dispose();
```

Dispose everything you create. A controller and a FocusNode belong to your State object, and leak if forgotten.

FORMS

Errors people can act on

Errors under the field

The message sits where the mistake is. Nobody hunts for the wrong field.

Server errors

”Email already registered” is not a validator result. Show it near the button.

Disable submit in flight

Keep a bool in the bloc state. A double tap must not create two accounts.

Keep what they typed

On failure, keep every field. Retyping after a network blink loses users.

Scroll to the error

On a long form the field can be off screen. `Scrollable.ensureVisible` helps.

Say what succeeded

A form that submits silently looks broken. One SnackBar is enough.

PART 3 · MOTION

Animation you get for free

Implicit animations, Hero, and knowing when to stop

The implicit animation family

AnimatedContainer

Change size, color, padding or decoration and it tweens between old and new.

AnimatedOpacity

Fade in or out. An opacity of zero still lays out and still hits tests.

AnimatedSwitcher

Cross-fades between two children when the child's key changes.

AnimatedAlign

Move a child inside its parent. So does AnimatedPadding.

AnimatedDefaultTextStyle

Grow, shrink or recolor text as a state changes.

TweenAnimationBuilder

Any value you can tween, animated once, with no State of your own.

MOTION

Two widgets, one state change

```
const d = Duration(milliseconds: 200);   // one place, whole app
AnimatedContainer(
  duration: d, curve: Curves.easeOut,
  color: done ? cs.primaryContainer : cs.surface,
  padding: EdgeInsets.all(done ? 8 : 16),
  child: const TodoTitle(),
);
AnimatedSwitcher(duration: d,
  child: loading ? const Spinner(key: ValueKey('spin'))
                 : TodoList(key: ValueKey(items.length)));
```

The key is what makes AnimatedSwitcher work. Without a different key, Flutter reuses the same element and there is nothing to cross-fade.

MOTION

Hero: one element across two routes

```
// List screen
Hero(
  tag: 'todo-${todo.id}',
  child: TodoThumbnail(todo),
);

// Detail screen, same tag
Hero(tag: 'todo-${todo.id}', child: big);
```

The tag is the contract.
Flutter finds the widget carrying the same tag on both routes and flies it between the two positions.
A tag must be unique on each screen. Use the record's id, not the list index.

Hero borrows the route transition. No controller, no duration. The only thing you own is the tag, and duplicate tags throw.

When to stop animating

```
if (MediaQuery.of(context).disableAnimations) return child;
```

LENGTH	USE IT FOR	SIGN YOU GOT IT WRONG
100 to 150 ms	a ripple, a checkbox, a fade	the change is invisible
200 to 300 ms	cards, sheets, list items	the app feels slow to a fast user
300 to 400 ms	a full route transition	the user waits for the animation

Motion explains a change, it does not announce it. If the user notices the animation instead of the result, it is too long. Never block input until it ends.

PART 4 · REACH

Accessibility and localization

Semantics, contrast, tap targets, text scaling, ARB files

ACCESS

Six things that lock people out

Unlabeled controls

An icon button with no label is announced as ”button” and nothing else by TalkBack and VoiceOver.

Low contrast

Gray on gray fails outdoors, on a cheap panel, and for anyone over forty.

Tiny targets

A 24 dp icon is a 24 dp target unless you say otherwise. Fingers are not cursors.

Fixed heights

A row with a hard-coded height overflows the moment the user enlarges text.

Color as the only signal

Red text alone does not say ”error” to a person who cannot see red. Add an icon or a word.

No focus order

On a keyboard or a switch device, tab order is your layout order. Check it once.

ACCESS

Semantics: what the screen reader says

```
IconButton(
  icon: const Icon(Icons.delete),
  tooltip: 'Delete todo',   // also a label
  onPressed: _delete,
);

Semantics(
  label: '3 of 7 done',
  child: ExcludeSemantics(child: chart),
);
```

Describe the meaning, not the pixels.
A chart is a picture to the framework. `Semantics` replaces it with the sentence a person needs. `MergeSemantics` joins a row into one announcement. `liveRegion: true` makes a status line speak when it changes.

The framework guesses, and you confirm. Flutter builds a semantics tree on its own. Your job is every place where the pixels do not carry the meaning.

ACCESS

Contrast, targets, and text that grows

RULE	TARGET	HOW TO SATISFY IT
Body text contrast	4.5 to 1	an on role over its own surface role
Large text and icons	3 to 1	the same, plus a design-tool check
Tap target	48 dp, 44 pt	a minimum button size in the theme
Text scaling	up to 2 times	no fixed heights, let rows wrap
Focus order	follows reading	widget order equals visual order

Test with the phone’s own settings. Set the font size to the largest step, turn a screen reader on, and open every screen. Two minutes, and it finds most of these.

LOCALE

Strings live in ARB files, not in widgets

```
# pubspec.yaml
flutter:
  generate: true

# l10n.yaml
arb-dir: lib/l10n
template-arb-file: app_en.arb
output-localization-file: app_localizations.dart
```

Add `flutter_localizations` and `intl` as dependencies. Building the app regenerates the `AppLocalizations` class from the ARB files. Pass `AppLocalizations.localizationsDelegates` and `supportedLocales` to `MaterialApp`, then read a string with `AppLocalizations.of(context)`.

LOCALE

Plurals, dates, and right to left

```
// lib/l10n/app_en.arb
"todoCount": "{count, plural, =0{No todos} =1{1 todo} other{{count} todos}}",
"@todoCount": { "placeholders": { "count": { "type": "int" } } }

// In the widget
Text(AppLocalizations.of(context)!.todoCount(items.length));
Text(DateFormat.yMMMd(locale).format(todo.due));

// Right to left: start and end, never left and right
padding: const EdgeInsetsDirectional.only(start: 16),
```

Some languages have more than two plural forms, which is why you never build a sentence by concatenation.

PART 5 · TESTING

Proving it still works

Unit, widget, bloc, golden and integration tests

TESTING

The pyramid, and what each layer buys

LAYER	RUNS ON	SPEED	WHAT IT PROVES
Unit	the Dart VM, no UI	milliseconds	a validator, a model, a repository
Widget	a headless tree	milliseconds	a screen builds and reacts to a tap
Bloc	the Dart VM	milliseconds	an event produces these states
Golden	a headless tree	fast	the pixels did not change by accident
Integration	a device or emulator	seconds	the whole app, plugins and network

Write many cheap tests and a few expensive ones. Every layer catches a bug the layer below cannot see, and costs more to write and to run.

TESTING

Unit tests: the cheapest ones first

```
// test/validators_test.dart
void main() {
  group('validateEmail', () {
    test('rejects an empty value', () {
      expect(validateEmail(''), 'Enter your email');
    });
    test('accepts a normal address', () {
      expect(validateEmail('ana@example.com'), isNull);
    });
  });
}
```

Files end in `_test.dart` and live under `test/`. Run them all with `flutter test`.

TESTING

Widget tests: testWidgets, find, pump

```
testWidgets('adding a todo shows it in the list', (tester) async {
  await tester.pumpWidget(const MaterialApp(home: TodoScreen()));

  await tester.enterText(find.byType(TextField), 'Buy milk');
  await tester.tap(find.byIcon(Icons.add));
  await tester.pump();                          // build one new frame

  expect(find.text('Buy milk'), findsOneWidget);
  expect(find.byType(TodoTile), findsNWidgets(1));
});
```

Finders: `find.text`, `find.byType`, `find.byIcon`, `find.byKey`. Matchers: `findsOneWidget`, `findsNothing`, `findsNWidgets(n)`.

pump, pumpAndSettle, and the trap

```
await tester.pump();
// one frame

await tester.pump(
  const Duration(milliseconds: 300));
// advance time by 300 ms

await tester.pumpAndSettle();
// frames until no animation is left
```

Test time does not pass on its own.
Nothing animates, no timer fires and no Future completes until you pump. That is a feature: the test is deterministic.

⚠ Watch out

`pumpAndSettle` never returns if something animates forever, such as a spinner. Pump a fixed duration instead.

Faking the outside world with mocktail

```
class MockTodoRepository extends Mock implements TodoRepository {}

late MockTodoRepository repo;

setUp(() {
  repo = MockTodoRepository();
  when(() => repo.load()).thenAnswer((_) async => [Todo(id: 1)]);
});

test('the cubit asks the repository once', () async {
  await TodoCubit(repo).load();
  verify(() => repo.load()).called(1);
});
```

Testing a bloc: back to Lecture 6

```
blocTest<TodoBloc, TodoState>(
  'emits loading then loaded when TodosRequested is added',
  build: () => TodoBloc(repo),          // repo is a mock, see previous slide
  act: (bloc) => bloc.add(const TodosRequested()),
  expect: () => [
    const TodoState.loading(),
    TodoState.loaded([Todo(id: 1)]),
  ],
);
```

This is why state classes carry value equality. `expect` compares the emitted states to your list. Without Equatable, two identical states are not equal and the test fails.

TESTING

Golden tests: the pixels themselves

```
testWidgets('TodoTile matches its golden', (tester) async {
  await tester.pumpWidget(wrap(const TodoTile(title: 'Buy milk')));
  await expectLater(
    find.byType(TodoTile),
    matchesGoldenFile('goldens/todo_tile.png'),
  );
});
```

Create or update the reference images with `flutter test --update-goldens`, then review them in the pull request like any other change.

Commit the golden files. The diff in the review is a picture, which is the only test output a designer can read.

TESTING

integration_test, and running all of it in CI

```
flutter analyze
flutter test                        # unit, widget, bloc, golden
flutter test integration_test       # on a booted device or emulator
```

An integration test starts with `IntegrationTestWidgetsFlutterBinding.ensureInitialized()` and then uses the same `testWidgets` API, except that the real app runs on a real device with real plugins.

Green on your laptop proves nothing. Run analyze and test on every pull request. A branch that fails the checks cannot be merged. The pipeline itself is Lecture 14.

RECAP

Three things to remember

1. **Declare colors and type once.** A widget that reads a role instead of a hex value already supports dark mode, large text and a rebrand.
2. **Polish is mostly restraint.** Short animations, honest error messages, a 48 dp target, and a screen that survives being made twice as wide.
3. **Tests are how a refactor stays cheap.** Many unit and widget tests, a few golden and integration tests, all of them run by CI.

FURTHER READING

docs.flutter.dev/ui/design/material · Accessibility and i18n · Testing overview

Questions?

dinu_stefan.rusu@upb.ro · Microsoft Teams: course channel